

# Succinct Rabung certificates and recurrence-closed lower bounds for van der Waerden numbers

Brice Pouly

**Qualification status (28 August 2026).** This is a private prepublication draft, not the citable release. Open gates include the final 13 claim manifests, 515-chunk campaign recovery, ordered-prime audit, CUDA qualification, four external certificate rechecks, outbound licences, and DOI/tag freeze, as recorded in `STATUS.json`. Numerical statements below report the current computation; release-level reproducibility claims remain conditional on the full ledger.

## Abstract

The registered numerical result comprises nine direct Rabung bounds for  $W(3, k)$ ,  $17 \leq k \leq 25$ , headed by  $W(3, 17) > 31\,385\,622\,833$ . Conditional on the final fail-closed verification manifests, five of these are the strongest bounds obtained for  $17 \leq k \leq 21$  from an explicitly listed, dated corpus closed under the Blankenship–Cummings–Taranchuk and Xu recurrences; the other four improve the direct Rabung values for  $22 \leq k \leq 25$  but not the best general bounds in that corpus. Four archived two-colour primes due to Monroe also yield, in particular,  $W(3, 25) > 628\,673\,328\,287$  and  $W(3, 28) > 2\,159\,051\,058\,266$ . The GPU campaign only discovered candidate triples. For each triple  $(p, r, k)$ , a stand-alone, separate-source CPU verifier reconstructs the finite hypotheses of Rabung’s theorem; a fail-closed release artifact binds its code, commands and raw outputs. A secondary, model-conditional holdout comparison is reported separately in an appendix.

## 1 Introduction

How far can a large search in the power-residue family push finite lower bounds for van der Waerden numbers, and how can its output remain checkable without trusting the search engine? The two questions have different answers. A GPU scan can discover useful primes quickly; the durable mathematical object is instead the succinct triple  $(p, r, k)$ , whose finite Rabung hypotheses can be re-established from scratch by a separate-source CPU verifier. Only after that pointwise certification do we use recurrence closure to decide whether a direct-method improvement is also the strongest general bound in a dated source corpus.

For integers  $r \geq 2$  (colours) and  $k \geq 3$ , the van der Waerden number  $W(r, k)$  is the least  $N$  such that every  $r$ -colouring of  $\{1, \dots, N\}$  contains a monochromatic arithmetic progression of length  $k$ . We put the number of colours first. Monroe [8] uses the reverse convention  $W(k, r)$ , while Fox and Hunter [3] write  $w(k; r)$ .

The registered release claim contains nine direct three-colour Rabung certificates, for  $17 \leq k \leq 25$ . The headline candidate is

$$W(3, 17) > 31\,385\,622\,833,$$

from  $p = 1\,961\,601\,427$ , a factor 2.02 over Monroe’s published bound. Conditional on final acceptance of the thirteen claim manifests, recurrence closure leaves the direct bounds as the

strongest values located in the registered corpus for  $17 \leq k \leq 21$ . For  $22 \leq k \leq 25$  they remain improvements within the direct Rabung family but are dominated as general bounds. This distinction between the strongest direct Rabung bound located in the dated corpus and its strongest general consequence is part of the claim, not a bibliographic afterthought.

Two mechanisms make that distinction necessary. Rabung’s power-residue construction [12] turns a suitable prime  $p$  into the direct bound

$$W(r, k) > (k - 1)p + 1.$$

Blankenship, Cummings and Taranchuk [1, Theorem 2.1], by contrast, prove the finite constructive recurrence

$$W(r, k) > q(W(r - \lceil r/q \rceil, k) - 1), \quad (1)$$

where  $q$  is the largest prime not exceeding  $k$ . In particular, a documented strict bound  $W(2, k) > B$  implies  $W(3, k) > qB$ . Xu’s concatenation recurrence [14] supplies another closure operation. We therefore compare bounds only after a machine-readable registry of published, archived, and current inputs has been closed under both recurrences. Section 2 states Xu’s formula, the strict-integer convention and the finite corpus policy explicitly. The recent superexponential three-colour result of Fox and Hunter [3] and the asymptotic two-colour bound  $w(k) \geq (1 - o(1))k2^{k-1}$  of Campos, Fox and Schildkraut [2] are separate asymptotic developments; neither supplies a replacement numerical entry for the finite range compared here.

The historical campaign was configured to scan every prime in  $[9.7 \times 10^8, 2 \times 10^9]$  and to compute the active profiles with  $r \leq 9$ . Monroe’s article describes a systematic scan limit of  $9.5 \times 10^8$ , while its accompanying table lists the larger valid prime 969 347 371; we use the tabled value as the numerical hand-off. Recovery of the complete checkpoint and ordered-prime evidence remains a release gate. That recovery supports claims about exhaustive coverage of the window and the empirical event list; it is logically separate from pointwise verification of any particular triple.

Four additional registered triples are two-colour primes listed in an archived phase of Monroe’s project [7]. Retained historical runs report their acceptance by a separate-source checker, and the final raw manifests remain open. Priority for the primes is Monroe’s. Applying (1) to these inputs gives the three-colour consequences for  $25 \leq k \leq 28$  displayed in Section 2. The archive states that the historical phase tested primes to four billion [9]; our checker establishes the validity of the listed triples, not the exhaustiveness of that historical scan.

Computational power-residue lower bounds and cyclic-zipper extensions have previously been pursued by Herwig, Heule, van Lambalgen and van Maaren [4], Rabung and Lotts [11], Liang, Xu, Shao and Xiu [5], and Monroe [8]. Relative to that lineage, our contribution is the combination of new direct three-colour triples, a recurrence-closed and provenance-bearing comparison, and a stand-alone verification path whose arithmetic and source are separate from the GPU engine. We do not claim global optimality, exhaustive coverage beyond the stated window, first discovery of Monroe’s archived primes, or a universal density law.

Section 2 states the registered numerical results, the recurrence closure and their exact scope before the technical inventory. Section 3 gives the Rabung criterion, proves the operational form and explains the stand-alone certificate check. Section 4 describes discovery and the evidence needed for campaign coverage. We conclude with the scientific boundary and release contract. The reconstructed holdout record and its model-conditional analysis are separated into Appendices A–C.

## 2 Main results and exact scope

The computational source of truth registers exactly thirteen direct Rabung claims: nine three-colour triples for  $17 \leq k \leq 25$  and four two-colour triples for  $25 \leq k \leq 28$ . In this private build they retain candidate status because the final fail-closed manifests have not yet been populated. The release build promotes them to the following numerical theorem only when the exact set of thirteen accepted manifests is present.

**Validated-claim table unavailable in this build.** The numerical triples below are the registered release claims, but the canonical theorem and audit table appear only when the exact set of thirteen fail-closed claim manifests is available and validated. This placeholder is not evidence and cannot close the claim gate.

### 2.1 Three colours: direct and recurrence-closed bounds

The historical scan produced nine candidate direct Rabung rows. They involve only three distinct primes. By Lemma 3.2, the certificate (1 999 999 927, 3, 19) implies all six rows with the same prime and  $20 \leq k \leq 25$ . Thus only seven triples in the thirteen-row theorem are mathematically primitive: three at three colours and the four two-colour triples below. The release artifact nevertheless re-evaluates the complete criterion at all thirteen registered triples as deliberate audit redundancy; the six repeated checks are neither mathematically necessary nor counted as independent implementation paths.

The headline value

$$W(3, 17) > 31\,385\,622\,833$$

comes from  $p = 1\,961\,601\,427$  and improves Monroe’s published 15 509 557 937 by a factor 2.02 [8]. A direct certificate is not automatically a general record: a published bound in another colour column may imply a stronger result through a recurrence.

Let  $q_k$  denote the largest prime not exceeding  $k$ . From Blankenship–Cummings–Taranchuk [1, Theorem 2.1],

$$W(3, k) > q_k(W(2, k) - 1).$$

Their theorem permits every prime  $q \leq k$ . For three colours,  $q = 2$  reduces to the one-colour case, while every  $q \geq 3$  uses the same two-colour input; choosing the largest such prime therefore loses no BCT consequence for the three-colour outputs considered here. If the registered input is the strict integer bound  $W(2, k) > B$ , then  $W(2, k) - 1 \geq B$ , and the generated consequence is exactly

$$W(3, k) > q_k B, \tag{2}$$

with no off-by-one ambiguity.

For completeness, write  $WR(s, k)$  for Xu’s ring van der Waerden number: the least  $N$  such that, for every  $n \geq N$ , every  $s$ -colouring of  $\mathbb{Z}/n\mathbb{Z}$  has a monochromatic  $k$ -term arithmetic progression. Xu’s concatenation theorem states that, for  $k \geq 3$ ,  $s, t \geq 2$ , and  $5 \leq n < WR(s, k)$  whose least prime divisor exceeds  $k$ ,

$$W(st, k) \geq n(W(t, k) - 1) + 1. \tag{3}$$

Our registry stores strict integer bounds. Every ring witness  $R$  used here has least prime divisor greater than  $k$ , a precondition checked by the closure tests. Hence inputs  $WR(s, k) > R$  and

$W(t, k) > B$  permit  $n = R$  and give exactly  $W(st, k) > RB$ , with no rounding or off-by-one convention left implicit.

The baseline program iterates the registered BCT specialization and (3) to a fixed point. Its finite scope is  $17 \leq k \leq 28$  and at most four colours. At each step it retains exact integer bounds and a full provenance path. Candidates are compared first by bound; numerical ties use the deterministic order direct Rabung, published or archived seed, BCT, Xu, one-colour exact, and finally the hash-derived node identifier. These tie-breaks affect provenance only. Since both operators are monotone in their input bounds, a dominated parent cannot improve a descendant. Iteration stops when no winning node changes.

The comparison corpus was frozen on 28 August 2026 and consists exactly of the seeds listed in `audit/baselines.json`: Monroe’s published ordinary and ring tables, the immutable direct tally distributed with his code, the four archived phase-2 rows credited to Monroe, and the thirteen claims of this work. Published entries require a stable primary locator. An unpublished or archived entry is included only with an immutable source identity and explicit priority attribution; a merely announced value is not silently promoted to a seed. The generated output embeds this catalogue, SHA-256 hashes of both registries, and the winning provenance paths. Accordingly, “strongest” below always means strongest consequence of these explicitly listed inputs, not global optimality over an unspecified literature corpus.

| $k$ | direct $p$    | direct bound   | best closed bound | method                               |
|-----|---------------|----------------|-------------------|--------------------------------------|
| 17  | 1 961 601 427 | 31 385 622 833 | 31 385 622 833    | direct Rabung                        |
| 18  | 1 999 994 851 | 33 999 912 468 | 33 999 912 468    | direct Rabung                        |
| 19  | 1 999 999 927 | 35 999 998 687 | 35 999 998 687    | direct Rabung                        |
| 20  | 1 999 999 927 | 37 999 998 614 | 37 999 998 614    | direct Rabung                        |
| 21  | 1 999 999 927 | 39 999 998 541 | 39 999 998 541    | direct Rabung                        |
| 22  | 1 999 999 927 | 41 999 998 468 | 49 058 715 046    | BCT, $q = 19$ (published $W(2, k)$ ) |
| 23  | 1 999 999 927 | 43 999 998 395 | 142 741 265 701   | BCT, $q = 23$ (published $W(2, k)$ ) |
| 24  | 1 999 999 927 | 45 999 998 322 | 252 542 143 876   | BCT, $q = 23$ (published $W(2, k)$ ) |
| 25  | 1 999 999 927 | 47 999 998 249 | 628 673 328 287   | BCT, $q = 23$ (archived input)       |
| 26  | –             | –              | 1 221 912 599 848 | BCT, $q = 23$ (archived input)       |
| 27  | –             | –              | 2 068 976 009 617 | BCT, $q = 23$ (archived input)       |
| 28  | –             | –              | 2 159 051 058 266 | BCT, $q = 23$ (archived input)       |

Table 1: Direct three-colour Rabung certificates and the best general lower bounds obtained after closing the registered baselines under the published Blankenship–Cummings–Taranchuk and Xu recurrences. A dash means that the GPU campaign made no direct three-colour claim for that length. For  $k = 22, 23, 24, 25$  the direct certificates are the strongest direct Rabung bounds located in the dated corpus but are not general records. Every entry is generated with a machine-readable provenance path in `audit/generated/bounds_closure.json`.

Thus, conditional on the direct-claim gate, the certificates for  $17 \leq k \leq 21$  give the strongest bounds located in the registered, recurrence-closed source corpus. For  $22 \leq k \leq 25$  the direct rows improve the direct Rabung family but are not best general bounds. In particular, the published two-colour values already dominate the direct rows at  $k = 22, 23, 24$ . No winning three-colour row in this finite scope uses Xu’s recurrence; it is retained because the comparison claim is closure under both registered operations.

## 2.2 Archived two-colour inputs and recurrence consequences

After the scan, we located a table from a second, unpublished phase of Monroe’s BOINC project (2021, two colours only) [7]. The project page states that this phase pushed the checked range from one billion to four billion [9]; the largest prime actually listed in the recovered table is 3 478 685 143. We credit the listed primes to Monroe and use our checker only to test whether the four displayed triples satisfy Rabung’s criterion.

| $k$ | prime $p$     | $W(2, k) >$    | method and provenance                                                  |
|-----|---------------|----------------|------------------------------------------------------------------------|
| 25  | 1 138 900 957 | 27 333 622 969 | archive and historical separate-source recheck; final manifest pending |
| 26  | 2 125 065 391 | 53 126 634 776 | archived Monroe prime; final manifest pending                          |
| 27  | 3 459 826 103 | 89 955 478 679 | historical no-Montgomery recheck; final manifest pending               |
| 28  | 3 476 732 783 | 93 871 785 142 | historical no-Montgomery recheck; final manifest pending               |

Table 2: Two-colour succinct Rabung certificates used as recurrence inputs. The method column separates priority for the archived prime from the separate-source recheck; final raw manifests remain pending.

For  $k = 25$ , even Monroe’s published input would give

$$W(2, 25) > 23\,003\,662\,489 \implies W(3, 25) > 529\,084\,237\,247.$$

The archived input accepted in retained historical checks,  $W(2, 25) > 27\,333\,622\,969$ , strengthens this to the first line below.

$$\begin{aligned} W(3, 25) &> 628\,673\,328\,287 \\ W(3, 26) &> 1\,221\,912\,599\,848 \\ W(3, 27) &> 2\,068\,976\,009\,617 \\ W(3, 28) &> 2\,159\,051\,058\,266 \end{aligned}$$

These four inequalities are theorem consequences of the corresponding two-colour inputs, not additional GPU discoveries. Priority for the primes remains Monroe’s. Xu’s recurrence is included in the automatic closure but does not improve any of these three-colour winners. Final release-level reproducibility remains conditional on the manifest protocol of Section 4.3.

For  $k = 25$  the campaign summaries identify the same last candidate in  $[9.7 \times 10^8, 2 \times 10^9)$  as the archived table; complete campaign recovery and ordered-prime identity remain release gates. For  $k = 26, 27, 28$  our campaign-range candidates are weaker, and the displayed archive primes were checked directly in retained historical runs. None of this establishes that Monroe’s phase exhaustively tested every prime up to four billion: that is an attributed archival statement, not a consequence of the recovered table.

The source is the Internet Archive capture of 28 April 2022 of <http://vdownumbers.org/bounds.txt> [7]; its header dates the file to 18 August 2021. The release records the immutable capture URL, both dates, source-repository blob identity, and SHA-256 in `publication/baseline-provenance.json`. It does not redistribute the third-party table absent permission.

Once the final manifests and external execution records are complete, the defensible novelty claim is a separately auditable re-verification with a public stand-alone checker and complete logs—not first discovery of the primes and not a newly audited claim of historical scan coverage.

### 3 The Rabung criterion and certificates

#### 3.1 Power-residue colorings

The certification argument has three layers. Rabung’s theorem turns a finite criterion into a lower bound; Proposition 3.3 proves the equivalent operational form used here; and a stand-alone, separate-source checker re-establishes that criterion for each reported triple. The GPU search of Section 4 discovers candidate triples but is not a premise of this implication.

An *explicit certificate* for  $W(r, k) > n$  is an  $r$ -colouring of  $\{1, \dots, n\}$  with no monochromatic  $k$ -term progression. The large claims instead use a *succinct Rabung certificate*  $(p, r, k)$ , from which the checker reconstructs the hypotheses of the construction theorem. It does not enumerate every progression in the resulting colouring.

Let  $p$  be a prime with  $p \equiv 1 \pmod{r}$  and  $p > k$ , let  $g$  be a primitive root modulo  $p$ , and let  $\text{ind}(x)$  denote the discrete logarithm of  $x$  to base  $g$ . The *power-residue coloring* is

$$c(x) = \text{ind}(x) \bmod r, \quad x \in \{1, \dots, p-1\},$$

i.e. the partition of  $\mathbb{Z}_p^*$  into the  $r$  classes of  $r$ -th power residues. Its key property is multiplicativity:  $c(xy) = c(x) + c(y) \pmod{r}$ . Consequently, dilating a progression  $\{a, a+d, \dots, a+(k-1)d\}$  with  $p \nmid d$  by  $u = d^{-1} \bmod p$  shifts all colors by the constant  $c(u)$  and maps the progression to  $k$  *consecutive* residues modulo  $p$ . The search for monochromatic progressions therefore reduces to a linear scan of the run lengths of the sequence  $c(1), c(2), \dots, c(p-1)$ , plus a boundary condition at the multiples of  $p$ ; this reduction from an  $O(p^2/k)$  test to an  $O(p)$  test per prime is what makes a mass scan possible.

#### 3.2 Rabung’s theorem and the operational criterion

Rabung [12] proved that the extension  $\mathcal{V}'$  of the power-residue coloring to  $[0, (k-1)p]$ , with the multiples of  $p$  colored by any non-constant assignment, is free of monochromatic  $k$ -term progressions *if and only if*

- (a) no  $k$  consecutive positions in  $[1, p-1]$  receive the same color ( $\max \text{run} < k$ ); and
- (b) a boundary condition holds. Put  $\varepsilon := c(-1) = c(p-1) \in \{0, r/2\}$ , where the second value is possible only for even  $r$ . In Rabung’s multiplicative notation the corresponding sign is  $+1$  when  $\varepsilon = 0$  and  $-1$  when  $\varepsilon = r/2$ . If  $\varepsilon = 0$  the integers  $1, 2, \dots, \lceil (k-1)/2 \rceil$  must not all lie in the same class, while if  $\varepsilon = r/2$  the same is required of  $1, 2, \dots, k-1$ . A *singleton* counts as “all in the same class”, and so fails (b).

If  $r$  is odd,  $(p-1)/r$  is even and hence  $\varepsilon = 0$  necessarily.

**Remark 3.1.** Both details of (b) matter. Rabung’s bracket here denotes the ceiling. Replacing it by a floor, or treating a singleton as vacuously satisfying the condition, yields a criterion that is *not* equivalent to Rabung’s. Explicit counterexamples occur already at  $p = 13$ ,  $r = 2$ ,  $k = 4$  for the floor variant and at  $p = 5$ ,  $r = 2$ ,  $k = 3$  when a singleton is mishandled. The ceiling

and floor coincide only for odd  $k$ . Both failure modes occurred during development and were detected by comparison with the finite oracle described below.

Since  $[0, (k-1)p]$  contains  $(k-1)p+1$  integers, a prime satisfying (a) $\wedge$ (b) yields

$$W(r, k) > (k-1)p + 1. \quad (4)$$

We call such a prime *valid* for  $(r, k)$ .

Our implementations test an equivalent form of (b), denoted  $(B^*)$ , which is uniform in the parity of  $k$  and the value of  $\varepsilon$ : for every shift  $j \in \{0, \dots, k-1\}$ , the  $k-1$  character positions  $\{-j, \dots, -1, 1, \dots, k-1-j\}$  of a  $k$ -window centered on a multiple of  $p$  must not all receive the same color. The equivalence rests on the multiplicative structure: if a single shifted window is monochromatic, its dilates realize every color at the central position (surjectivity of  $c$  requires exactly  $p \equiv 1 \pmod{r}$ ), so the joker position cannot be colored at all; otherwise any non-constant coloring of the multiples of  $p$  works. Proposition 3.3 gives the mathematical proof; the finite comparison below audits the implementation rather than replacing that proof.

**Lemma 3.2** (Monotonicity at fixed prime). *Let  $r \geq 2$ , let  $3 \leq k \leq \ell < p$ , and let  $p \equiv 1 \pmod{r}$  be prime. If  $p$  satisfies (a)  $\wedge$   $(B^*)$  for  $(r, k)$ , then it satisfies the same criterion for  $(r, \ell)$ . Consequently*

$$W(r, \ell) > (\ell-1)p + 1.$$

*Proof.* It is enough to pass from  $k$  to  $k+1$ . A monochromatic run of length  $k+1$  would contain one of length  $k$ , so condition (a) persists. Write

$$S_j^{(m)} = \{-j, \dots, -1\} \cup \{1, \dots, m-1-j\}, \quad 0 \leq j \leq m-1.$$

For  $0 \leq j \leq k-1$  we have  $S_j^{(k)} \subset S_j^{(k+1)}$ , and  $S_{k-1}^{(k)} \subset S_k^{(k+1)}$ . Thus every boundary string for length  $k+1$  contains one already known to be non-monochromatic, so  $(B^*)$  persists. Iteration gives the claim.  $\square$

### 3.3 Equivalence of the operational criterion

The scan tests the criterion (a)  $\wedge$   $(B^*)$  of Section 3 rather than Rabung's (a)  $\wedge$  (b) verbatim. The two are equivalent, and since every bound in this paper rests on that equivalence we prove it here rather than rely on the exhaustive machine comparison of Section 3, which we regard as a check on the *implementation*, not as a substitute for a proof.

**Setting.** Let  $p \equiv 1 \pmod{r}$  with  $p > k \geq 3$ , let  $g$  be a primitive root, and colour  $x \in [1, p-1]$  by  $c(x) = \text{ind}_g(x) \pmod{r}$ ; this is the power-residue colouring, and  $c(xy) = c(x) + c(y)$ . Extend it to  $[0, (k-1)p]$  by reducing modulo  $p$  and colouring the multiples of  $p$  (the *jokers*) by an arbitrary map  $\vartheta' : \{0, p, \dots, (k-1)p\} \rightarrow \mathbb{Z}_r$ . Put  $\varepsilon := c(-1) = c(p-1)$ , which is 0 if  $-1$  is an  $r$ -th power residue and  $r/2$  otherwise (the latter requires  $r$  even).

An arithmetic progression of length  $k$  and common difference  $d$  is monochromatic iff its terms all receive one colour. Progressions avoiding the jokers are handled by (a): by complete multiplicativity of  $c$ , the progression  $a, a+d, \dots, a+(k-1)d$  reduces, after dividing by  $d$  (which permutes the classes by the constant  $c(d)$ ), to the window  $\{a/d, a/d+1, \dots, a/d+k-1\}$  of  $k$  consecutive residues. Hence *some* joker-free progression is monochromatic iff *some* window of  $k$  consecutive residues in  $[1, p-1]$  is, which is exactly the negation of (a).

**The joker condition.** It remains to treat the progressions that meet a joker. First bound the common difference: a  $k$ -term progression inside  $[0, (k-1)p]$  has  $(k-1)d \leq (k-1)p$ , so  $1 \leq d \leq p$ , and the case  $d = p$  occurs for exactly one progression, namely  $0, p, 2p, \dots, (k-1)p$ , which consists *entirely* of jokers. That progression is monochromatic if and only if  $\vartheta'$  is constant—it is precisely the reason Rabung’s statement requires a *non-constant* joker colouring, and any non-constant  $\vartheta'$  breaks it. For the remaining case  $0 < d < p$ : since the progression has  $k < p$  terms, two of them congruent to 0 modulo  $p$  would force  $p \mid (i' - i)d$  with  $0 < |i' - i| < k < p$  and  $p \nmid d$ , which is impossible; so the progression meets at most one multiple of  $p$ . Say it is the  $(j+1)$ -st term,  $0 \leq j \leq k-1$ . Dividing by  $d$  as before, the progression becomes the *shifted string*

$$S_j = \{-j, \dots, -1\} \cup \{1, \dots, k-1-j\} \pmod{p},$$

centred on the joker, together with the joker itself.

**Proposition 3.3.** *Fix  $p \equiv 1 \pmod{r}$  and  $k$ . The colouring above is free of monochromatic  $k$ -term progressions for some choice of  $\vartheta'$  if and only if  $(a) \wedge (B^*)$  holds, where*

$$(B^*) : \quad \text{no shifted string } S_j \text{ } (0 \leq j \leq k-1) \text{ is monochromatic.}$$

Moreover, when  $(a) \wedge (B^*)$  holds, every non-constant  $\vartheta'$  works; and when it fails, no  $\vartheta'$  works. In particular the choice of  $\vartheta'$  is immaterial, as asserted by Rabung.

*Proof.* Suppose some  $S_j$  is monochromatic, of colour  $w$  say.

*The lift.* We first check that every residue class of  $d$  is realised by a genuine progression inside  $[0, (k-1)p]$  through a joker. Fix a representative  $d \in [1, p-1]$  and take the joker to be  $jp$  and the first term to be

$$a = j(p-d).$$

Then  $a + jd = jp$ , so the  $(j+1)$ -st term is the joker; the first term  $a = j(p-d) \geq 0$  since  $d < p$ ; and the last term is

$$a + (k-1)d = jp + (k-1-j)d \leq jp + (k-1-j)(p-1) \leq (k-1)p,$$

with equality in the last step only at  $j = k-1$ , where that last term is the joker  $jp$  itself; so all  $k$  terms lie in  $[0, (k-1)p]$ . The non-joker terms are  $jp + id$  for  $i \in S_j$ , whose residues are  $id \pmod{p}$ ; hence their colours are  $c(i) + c(d) = w + c(d)$ .

*The all-or-nothing mechanism.* As  $d$  ranges over  $(\mathbb{Z}/p)^*$ , the value  $w + c(d)$  ranges over *all* of  $\mathbb{Z}_r$  (because  $c$  is onto), and by the lift each such value is attained by an admissible progression through the *same, fixed* joker  $jp$ . Hence for each colour  $u \in \mathbb{Z}_r$  there is a progression through  $jp$  whose other  $k-1$  terms are all of colour  $u$ ; assigning  $\vartheta'(jp) = u$  makes it monochromatic. Every value of  $\vartheta'$  at the single joker  $jp$  is thus forbidden, so no choice of  $\vartheta'$ —constant or not—avoids a monochromatic progression. Conversely, if no  $S_j$  is monochromatic then no progression with  $d < p$  through a joker has all its other  $k-1$  terms of a single colour, so no assignment of  $\vartheta'$  can complete one; the only remaining progression is the all-joker one ( $d = p$ ), which any *non-constant*  $\vartheta'$  breaks; combined with (a) for the joker-free progressions, every non-constant  $\vartheta'$  works. The  $d = p$  case is exactly what consumes the non-constancy hypothesis. Thus  $(a) \wedge (B^*)$  is all-or-nothing over non-constant  $\vartheta'$ , which is why Rabung may state it without reference to the joker colouring.  $\square$

**Reduction of  $(B^*)$  to a single interval.** Since  $c(-x) = c(x) + \varepsilon$ , the string  $S_j$  is monochromatic iff the two intervals  $\{1, \dots, j\}$  and  $\{1, \dots, k-1-j\}$  are each monochromatic and their colours agree after the shift by  $\varepsilon$ .

*Case  $\varepsilon = 0$ .* Then  $c(-x) = c(x)$ , so  $S_j$  is monochromatic iff the interval  $\{1, \dots, \max(j, k-1-j)\}$  is. Minimising  $\max(j, k-1-j)$  over  $0 \leq j \leq k-1$  gives  $\lceil (k-1)/2 \rceil$ , attained at  $j = \lfloor (k-1)/2 \rfloor$ . Hence

$$(B^*) \iff \{1, 2, \dots, \lceil (k-1)/2 \rceil\} \text{ is not monochromatic.}$$

A one-element interval *is* monochromatic, and therefore fails the condition; under the standing assumption  $k \geq 3$  this is the case  $k = 3$ , and it is the reason a singleton must be counted as “all in the same class”.

*Case  $\varepsilon = r/2$  ( $r$  even).* Now  $c(-x) = c(x) + r/2 \neq c(x)$ , so a two-sided string can never be monochromatic: only the one-sided strings  $j = 0$  and  $j = k-1$  survive, both giving the interval  $\{1, \dots, k-1\}$ . Hence

$$(B^*) \iff \{1, 2, \dots, k-1\} \text{ is not monochromatic,}$$

Unlike the length- $k$  prohibition of (a), this is a condition on an interval of length  $k-1$ , and is therefore *not* implied by (a): it is a genuine additional constraint, and on a longer interval than in the previous case.

**Remark 3.4.** Both refinements matter. Replacing  $\lceil \cdot \rceil$  by  $\lfloor \cdot \rfloor$ , or treating a singleton as vacuously satisfying the condition, gives a boundary condition that is not equivalent to (b). Counterexamples occur already at  $p = 13$ ,  $r = 2$ ,  $k = 4$  for the floor variant and at  $p = 5$ ,  $r = 2$ ,  $k = 3$  for the singleton convention; condition (a) can mask some such boundary errors.

This proposition is the mathematical bridge used for every one of the thirteen registered triples: once a verifier establishes  $(a) \wedge (B^*)$ , Rabung’s theorem yields the corresponding direct inequality.

### 3.4 Finite oracle audit of the implementation

We tested the implementation of  $(a) \wedge (B^*)$  against a frozen reference oracle that explicitly builds the coloring of  $[1, (k-1)p+1]$  and exhaustively checks all  $k$ -term progressions. One bookkeeping detail matters: Rabung’s construction lives on  $[0, (k-1)p]$ , and the oracle works on its translate by one—it colors  $x$  by the class of  $(x-1) \bmod p$  and places the jokers at  $x \equiv 1 \pmod p$ —so the two objects are literally the same coloring up to relabeling of positions, and freedom from monochromatic progressions is translation-invariant. A sweep over all primes  $p < 10^4$  with  $p \equiv 1 \pmod r$ ,  $p > k$ ,  $r \in [2, 6]$ ,  $k \in [3, 8]$  — 20 170 cases, of which 4 499 were valid according to the criterion — reported zero disagreements. A fresh staging replay reproduced those counts and its exact source identity, environment, and output streams are archived with this repository. Because that replay was not made from the final clean release candidate, the final CPU gate must still rerun it from the exact candidate commit and preserve durable raw outputs before the local tag is published. A stronger test enumerates all  $r^k$  colorings of the joker positions ( $p < 150$ ,  $r \leq 4$ ,  $k \leq 5$ ) and confirms both directions of the equivalence:  $(a) \wedge (b)$  holds if and only if some valid extension exists, and when it holds every non-constant joker coloring is valid. We emphasize the epistemic status: Rabung’s theorem is proved in [12]; the agreement of our operational form with the explicit oracle is exhaustively machine-checked on the stated domain.

### 3.5 Certificates and verification regimes

For the bounds announced in Section 2 the certificate length  $(k - 1)p + 1$  is of order  $3 \times 10^{10}$  to  $10^{11}$ , so running the naive quadratic oracle on the explicit certificate is not feasible. Our verification policy therefore distinguishes two regimes. On the finite validation domain actually reported above ( $p < 10^4$ ,  $k \leq 8$ , hence certificate length below  $7 \times 10^4$ ), the explicit certificate was checked directly by the reference oracle. We make no general feasibility claim for length  $10^7$ : a successful naive oracle must inspect

$$\sum_{d=1}^{\lfloor (n-1)/(k-1) \rfloor} (n - (k - 1)d)$$

progressions. For the large primes, the proof of (4) is Rabung’s theorem together with conditions (a)∧(b), and *these* are what we verify redundantly: the mirror, full-interval, and CPU-walk paths, followed by separate CPU implementations of the full criterion. Section 4.2 records which paths share source code or arithmetic; cross-agreement is not described as statistical or implementation independence.

At the large sizes of Section 2, distributing the full colouring would be neither feasible nor useful. The reproducibility package therefore distributes the triple  $(p, r, k)$ , from which the stand-alone, separate-source verifier `verify_claim` re-derives every required hypothesis. Its protocol is:

1. check that  $p$  is prime by deterministic Miller–Rabin on the first twelve prime bases 2, 3, 5, . . . , 37. Sorenson and Webster’s Theorem 1.1 gives

$$\psi_{12} = 318\,665\,857\,834\,031\,151\,167\,461 > 2^{64},$$

so this is exact through the unsigned 64-bit range [13], far beyond the  $p < 2^{32}$  actually used;

2. check the preconditions  $p \equiv 1 \pmod{r}$  and  $p > k$ ;
3. *find* a primitive root  $g$  by trial, certifying primitivity against the full trial-division factorization of  $p - 1$  (nothing about  $g$  is taken as input);
4. build the residue classes by modular exponentiation— $x^{(p-1)/r}$  compared against the powers of  $\zeta = g^{(p-1)/r}$ —using neither the discrete-logarithm walk nor the GPU pipeline;
5. test (a)∧(B\*) and print ACCEPT or REJECT.

The whole run takes  $O(p \log p)$  time and  $O(p)$  memory. Each claimed bound in this paper can thus be re-derived from the triple  $(p, r, k)$  alone—no generator, witness, or auxiliary data are part of the certificate—in about two to just over seven minutes on a single core (Table 3). In this precise sense, every claim is recheckable from its triple alone.

## 4 GPU discovery campaign

The campaign has one logical role: it generates candidate triples  $(p, r, k)$ . No lower bound follows from a scanner verdict alone. A retained candidate becomes a numerical claim only after the criterion is re-established through the verification protocol below; recovery of the complete campaign record is needed for coverage and density statements, but not for the pointwise validity of an accepted certificate.

| $p$           | $(r, k)$ | wall time | peak RAM | verdict |
|---------------|----------|-----------|----------|---------|
| 1 138 900 957 | (2, 25)  | 130 s     | 1.1 GB   | ACCEPT  |
| 1 961 601 427 | (3, 17)  | 229 s     | 1.9 GB   | ACCEPT  |
| 3 459 826 103 | (2, 27)  | 404 s     | 3.3 GB   | ACCEPT  |
| 3 476 732 783 | (2, 28)  | 425 s     | 3.3 GB   | ACCEPT  |

Table 3: Measured cost of the stand-alone, separate-source verifier on four representative certificates. The two high-range rows are cross-documented with the corrected-scanner and Montgomery-free checks required by the fail-closed claim manifest protocol. Those release manifests are not yet populated, so these historical timings are not themselves release evidence. Single-threaded on an AMD Ryzen 7 9800X3D under Ubuntu 26.04, `g++ 15.2.0 -O2 -std=c++17`. The time is modest; the memory is the real constraint, since the present implementation stores the colouring, i.e.  $O(p)$  bytes. A two-pass streaming variant with  $O(1)$  memory is possible and would be a worthwhile improvement, but it is not what we ran, and we report what we ran.

#### 4.1 Fused character evaluation

The scan kernel exploits the fact that the additive order of positions is free: we iterate  $x = 1, 2, \dots, p-1$  directly and evaluate the color of  $x$  on the fly by modular exponentiation, fused with the run-length bookkeeping. Nothing is sorted and no per-element value is materialized in memory. A single pass evaluates  $y = x^{(p-1)/m}$  with  $m = \text{lcm}\{r \leq 9 : r \mid p-1\} \leq 2520$ ; a table lookup of  $y$  among the  $m$ -th roots of unity gives  $\text{ind}(x) \bmod m$ , hence the color for every modulus  $r \leq 9$  simultaneously. Runs are summarized per segment by (prefix, suffix, best interior) triples combined with an associative, order-respecting reduction.

A second symmetry halves the work. Since  $c(p-x) = c(x) + c(-1)$ , the second half of the sequence is the reversal of the first half shifted by the constant  $c(-1)$ ; it suffices to scan  $x = 1, \dots, (p-1)/2$  and to splice the two halves at the center. We refer to this half-interval variant as the mirror scan.

#### 4.2 Redundant and separate-source validation paths

Correctness is supported by cross-agreement among redundant paths and by separate-source checkers rather than by trust in any single path. On all primes  $p < 105\,000$  (both residue classes of  $p \bmod 4$  and varied  $m$ ; the  $r$ -range of each comparison set is given with it):

- the sort-based GPU pipeline (an earlier, algorithmically distinct GPU design retained as a redundant path) agrees with the CPU reference on 50 105 comparisons ( $r \in [2, 6]$ ) with zero disagreements;
- the fused multi- $r$  kernel agrees with the CPU reference on 27 508 comparisons ( $r \in [2, 9]$ ) with zero disagreements (and the specialized  $r = 2, 3$  variant on a further 9 068);
- the mirror scan, the fused full scan, the sort-based pipeline, and the CPU reference agree on 33 327 comparisons over  $10^4$  primes with zero disagreements.

These comparison totals and the performance measurements below are historical pre-flight records. Their exact raw GPU outputs and benchmark logs must still be recovered from the campaign PC

and bound to the release candidate; the portable staging suite exercises the algorithms but does not treat these totals as release-qualified evidence. For  $r = 2$  a CPU Jacobi-symbol evaluation provides a further separate-arithmetic route. This redundancy is not decorative: during pre-flight testing the cross-check caught a genuine bug (a non-commutative run-summary operator fed to a reduction that assumes commutativity, producing phantom runs at  $p \sim 1.8 \times 10^9$ ); the repaired, order-respecting reduction is covered by a regression test that has run at every commit since.

The main paths have the following dependency structure; agreement between paths is interpreted in light of their shared source and arithmetic.

| path                | language | modular arithmetic     | class/run construction            | shared components                                    |
|---------------------|----------|------------------------|-----------------------------------|------------------------------------------------------|
| mirror V2a          | CUDA C++ | GPU Montgomery         | character, half interval          | same source and helpers as full B7                   |
| full B7             | CUDA C++ | GPU Montgomery         | character, full interval          | same source and helpers as V2a                       |
| sort-based GPU      | CUDA C++ | GPU Montgomery         | sort then run scan                | earlier GPU design; shares GPU Montgomery arithmetic |
| CPU walk            | C++ host | native 64-bit walk     | discrete logarithm, sequential    | same executable and host primitive-root code         |
| CPU criterion       | C++      | separate 128-bit path  | separate walk and (B*)            | separate source and binary                           |
| <b>verify_claim</b> | C++      | 128-bit exponentiation | roots of unity and full criterion | strongest source/arithmetic separation               |
| high-range witness  | C        | no Montgomery          | full sequential walk              | separate arithmetic; tests (a), not (B*)             |

Table 4: Dependency matrix for the verification paths. V2a and B7 are correlated redundant paths. The principal source- or arithmetic-separated checks are the separate CPU criterion, **verify\_claim**, and, for the high-range arithmetic defect, the Montgomery-free witness.

### 4.3 Claim-verification protocol

The computational source of truth registers exactly thirteen direct Rabung claims: nine three-colour triples for  $17 \leq k \leq 25$  and four two-colour triples for  $25 \leq k \leq 28$ . For each triple  $(p, r, k)$  the release protocol requires:

1. two executions of the structured triplet {mirror scan, full scan, CPU walk}, with internal profile equality and equality between executions;
2. two acceptances by the separate full CPU criterion;
3. acceptance by the stand-alone **verify\_claim**; and
4. for the two primes above the historical Montgomery-safety threshold, an additional full-interval witness that uses no Montgomery arithmetic.

The parser is fail-closed: it checks the process status, the exact  $p, r, k$ , all profile equalities, (a), (B\*), and the final verdict, and rejects any output containing **DIFF**, **FAIL**, **ECHEC**, or a negative verdict. Its digest binds the claim identity, profiles, criterion values, and final verdict. The manifest separately records source and binary hashes, compiler version, exact commands, and

hashes of every raw output. The final Markdown and L<sup>A</sup>T<sub>E</sub>X claim tables are generated only from the exact set of thirteen accepted manifests.

This rule is stricter than the historical transcript, whose max-run checksum did not bind  $p$  and whose pasted `verify_claim` outputs were sometimes truncated. Those files remain historical evidence, not the final source of truth. The expensive release run must populate the complete manifests before the candidate theorem of Section 2 can be promoted.

#### 4.4 Measured throughput

On the target hardware (one consumer RTX 5090, CUDA, 32-bit Montgomery arithmetic) the mirror multi- $r$  kernel was benchmarked at  $4.86 \times 10^{10}$  physical evaluations of  $x$  per second at  $p \approx 1.2 \times 10^9$ . Mirror reconstruction makes this equivalent to  $9.72 \times 10^{10}$  positions of the full sequence per second. The same run performed  $2.09 \times 10^{12}$  modular multiplications per second, 61% of the measured pure-multiplication plateau ( $3.40 \times 10^{12}$ ). Under the 431 W campaign power cap the observed kernel throughput was  $4.2\text{--}4.45 \times 10^{10}$  physical evaluations, or  $8.4\text{--}8.9 \times 10^{10}$  reconstructed full-sequence positions, per second. These are active-kernel rates. The ten-day campaign duration is wall time and also includes sieving, host work, checkpointing, cross-checks, and restarts; we do not quote a wall-clock modular-multiplication rate. Several plausible optimizations (e.g. a 2-adic factorization of the exponentiation) were implemented, measured, and rejected as memory-bound regressions.

These retained figures are historical engineering measurements, not a standardized performance study: the surviving record does not reconstruct a fixed warm-up, repetition count, dispersion statistic, clock state or complete power telemetry for every figure. They are not used in any mathematical claim. If they remain in the citable release, the accompanying CUDA artifact must separately record driver and toolkit versions, compiler flags, source and binary hashes, frequencies and power limit, warm-up, repetitions, and median and dispersion; otherwise only the qualitative campaign description will be retained.

#### 4.5 Campaign execution

According to the preserved campaign design and summaries, the production run was configured to scan every prime in  $[9.70 \times 10^8, 2.0 \times 10^9)$  (each prime tested for every  $r \leq 9$  dividing  $p - 1$ ), starting just above the largest published prime 969 347 371 of Monroe’s distributed scan [8], in a single multi- $r$  pass ( $r \leq 9$ ). (The narrow interval  $[969\,347\,371, 9.70 \times 10^8)$  was therefore not scanned; it is immaterial, since any valid prime in it would give a bound below every one we report.) The historical checkpoint reports that the range was processed in 515 restartable chunks of width  $2 \times 10^6$ , each with a fixed walltime cap. The registered revision map spans *eight* git revisions and *four* scanner-source fingerprints. Their exact allocation and the scientific effect of each transition are generated from the audit registry below. The (3, 17) max-run statistic and its target predicate are unchanged across all four fingerprints. The first functional expansion added only (2, 26)–(2, 28), whose early online tallies are therefore incomplete; the second added congruence counters and reproduced the previous target profiles and checksums in an old/new comparison. This supports code-path comparability for the retained (3, 17) counts, although version-stratified dispersion diagnostics remain distinct from that code audit. The historical journal also records automatic cross-checks every 50 chunks: nine completed samples agreed across the redundant scan paths and separate CPU checks, while a tenth timed out and was not counted. These samples do not turn the single production campaign into a duplicated

| source hash | chunks | revisions (number of chunks)                           | targets | change and transition check                                                                                                                                                                   |
|-------------|--------|--------------------------------------------------------|---------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 87f10166    | 1–7    | ebd54ea (7)                                            | 10      | Initial campaign fingerprint; fused and mirror max-run paths already active. Covered by the frozen pre-flight agreement suite.                                                                |
| 3598aea6    | 8–35   | 657a140 (10), 185b868 (18)                             | 10      | Added the point-verification mode and record reporting; the production target predicate was unchanged. Point claims were checked by mirror, full, CPU-walk, and separate verifiers.           |
| cca301fe    | 36–78  | 062cc3c (43)                                           | 13      | Added the three two-colour bonus targets; the already-computed $r=2$ max-run profile was reused. The new bookkeeping was exercised on [1026949000,1026950000); the (3,17) path was unchanged. |
| 3b5d1049    | 79–515 | 4f9e2cf (1), c3a9b9e (274), a93cec9 (72), 7c1ba34 (90) | 13      | Added aggregate congruence counters; later revisions changed the driver or theory files, not scan_gpu.cu. Old/new TARGET and CHECKSUM outputs agreed exactly and an end-to-end test passed.   |

Table 5: Scanner-source fingerprints and Git revisions used by the 515 campaign chunks. The registry also records the active target list and Git blob identity.

run. Checkpoint transaction mechanics are documented in `REPRODUCIBILITY.md`; the immutable checkpoint, transition outputs, chunk evidence, and ordered-prime stream remain release gates.

**Exhaustiveness of the prime enumeration.** The per-prime computation is validated by the redundancy above; the *enumeration* of primes deserves its own audit, since the density claims of Section B are claims about *all* primes in the window. The scanner enumerates primes with a plain in-process sieve per chunk; chunks are contiguous by construction ( $[\text{lo} + 2i \cdot 10^6, \text{lo} + 2(i+1) \cdot 10^6)$ ), and the checkpoint protocol resumes only at chunk boundaries, so interruptions can neither skip nor duplicate primes within the range. We verified the retained aggregate coverage counts against a separately written segmented sieve with no shared code: over the 498 chunks for which per-chunk histograms were logged (the histogram feature was added at an early restart, from  $p = 1.004 \times 10^9$  on), the campaign’s per-chunk counts of primes in every congruence class  $p \equiv 1 \pmod{r}$ ,  $2 \leq r \leq 9$ —3 984 counts in all—agree with the separate-source sieve *exactly*, with zero discrepancies (47 181 875 primes on those chunks). For the first 17 chunks, which predate the histogram feature, per-class counts were not logged during the campaign; that range,  $[9.70 \times 10^8, 1.004 \times 10^9)$ , was therefore re-scanned after the campaign in one aggregate invocation with the post-campaign audit build (commit c81383885bd9; full identities are in the registry). The eight the retained Git history places the separate-source counts before the commit recording completion of that invocation (commit 30e8b83df3c0). This establishes documentary order inside the preserved DAG, not an independently timestamped precommitment. A version-specific, fail-closed parser now checks the preserved transcript, whose SHA-256 is dff4e409...de184; it verifies every historical TARGET, candidate, COLQ, HISTO, and TOP row before comparing the eight histogram sums with those separately recorded counts.

The re-scan also lists exactly 405 run-valid (3, 17) candidates, consistently with its TARGET row and its  $r = 3$  histogram. It was one invocation over the whole interval, however, so it supplies

| $r$ | $\#\{p : p \equiv 1 \pmod{r}\}$ | $r$ | $\#\{p : p \equiv 1 \pmod{r}\}$ |
|-----|---------------------------------|-----|---------------------------------|
| 2   | 1 641 614                       | 6   | 821 005                         |
| 3   | 821 005                         | 7   | 273 695                         |
| 4   | 820 559                         | 8   | 410 485                         |
| 5   | 410 333                         | 9   | 273 955                         |

Table 6: Aggregate congruence-class totals from the post-campaign re-scan of the first 17 chunks. All eight equal the separate-source counts placed earlier in the retained Git history. This relative order is not an external timestamp. It is one aggregate comparison, not 17 per-chunk comparisons and not an ordered-prime identity test.

aggregate class totals rather than  $17 \times 8$  per-chunk cells. These audits establish exact agreement of the counts; counts alone do not establish identity of the ordered prime lists. The audit utility therefore also emits a canonical ordered stream for byte comparison and hashing. Populating its per-chunk identity manifest for all 515 chunks is a release gate, not a consequence inferred from the 3984 matching counts. The four historical scanner blobs contain a byte-identical 249-byte `sieve_primes` function (SHA-256 recorded in the registry). That function accepted an inclusive upper endpoint, whereas the audit stream is half-open; every campaign boundary is an even integer greater than two, hence composite, so the two conventions enumerate exactly the same primes on all 515 chunks. The current scanner makes the half-open convention explicit. In total the window  $[9.7 \times 10^8, 2 \times 10^9)$  contains 48 823 489 primes in the separate-source sieve. Across the retained 498-chunk histograms and the aggregate 17-chunk re-scan, the preserved outputs report corresponding per-class totals; full byte-level ordered-prime identity remains a release gate. The separate-source sieve counts 1 448 736 primes in  $[9.7 \times 10^8, 10^9)$ , hence 47 374 753 in  $[10^9, 2 \times 10^9)$ . This equals the externally published difference  $\pi(2 \times 10^9) - \pi(10^9) = 98\,222\,287 - 50\,847\,534$  from Oliveira e Silva’s table [10]; it checks the interval difference, not either absolute value separately.

The available summaries, deterministic chunk checksums, periodic cross-checks, and aggregate coverage comparison report no inconsistency, but they do not yet prove continuity without data loss. Release recovery must still substantiate checkpoint continuity, the chunk 7 reconstruction, and the raw inventory. The strongest candidate direct bounds located by the scan were therefore passed to Section 4.3; no bound was recorded from the scanning pass alone.

## 5 Conclusion

The scale of the search and the scale of the proof are deliberately different. The GPU campaign was used to locate candidate primes across a large interval, but the mathematical object supporting each direct bound is the succinct triple  $(p, r, k)$ . Its finite Rabung hypotheses can be re-established without trusting the search engine. The search discovers candidates; the pointwise certificate, Rabung’s theorem, and the recurrence calculation support the inequalities.

Subject to acceptance of the final thirteen claim manifests, the nine registered three-colour triples give direct lower bounds for  $W(3, k)$  throughout  $17 \leq k \leq 25$ . After closing the dated source registry under the Blankenship–Cummings–Taranchuk and Xu recurrences, the direct values are the strongest bounds located in that corpus for  $17 \leq k \leq 21$ . For  $22 \leq k \leq 25$  they remain improvements within the direct Rabung family but are not the strongest general bounds. The four archived two-colour triples and their priority remain Monroe’s. Once their final manifests pass, our contribution will be a separately auditable recheck and the recurrence

consequences drawn from those inputs, not their first discovery or a new audit of Monroe’s historical scan coverage.

The holdout comparison has a different status. It is an empirical check of three recorded fixed intensities against one run-valid (3, 17) trajectory in one finite window. It neither proves a density law nor strengthens the direct-bound argument. Its value is narrower: it records how the three fixed forecasts behaved in one repository-recorded holdout comparison, with the disjoint increments primary and the probabilistic readings conditional on the stated model.

The remaining release boundary is operational but scientifically material. The exact final candidate still requires clean CPU and document replays, thirteen complete claim manifests, read-only recovery and ordered-prime identity for all 515 campaign chunks, clean CUDA qualification, four external certificate rechecks, resolved outbound licences, and the immutable DOI/tag/archive freeze. Until the claim manifests pass, the registered inequalities retain candidate status; until the remaining gates pass, the package is not citable and its campaign-coverage claims are not release qualified. Even after release, the computation will not assert the absence of better Rabung certificates; nor will it claim that the reported values are globally optimal beyond the recurrence-closed corpus and methods explicitly compared here.

## 6 Data, code, and release status

The publication-oriented repository is [Vulkin-prog/vdw-rabung-bounds](#). It was exported under an explicit allowlist from the development repository at the immutable source commit `02796ae9e08da5b051a94b2d9001763908375f6e`. At the time of this revision it remains a private preparation repository, not a citable release. Its machine-readable status ledger leaves, among others, the claim, campaign-recovery, ordered-prime, CUDA, external-replication, rights, and DOI gates open. The step-by-step operator and archival procedures are maintained in `REPRODUCIBILITY.md`; here we distinguish the three levels of evidence needed to interpret the paper.

**Individual triples.** The registry `audit/claims.json` contains exactly thirteen succinct Rabung certificates  $(p, r, k)$ . A reader can check one triple without reproducing the campaign by rebuilding the stand-alone verifier, which re-establishes primality, the congruence and size preconditions, and conditions  $(a) \wedge (B^*)$ . Release-level evidence additionally binds each verdict to the exact source, binary, command, environment, and raw output in a fail-closed manifest. Those final manifests have not yet been populated: the placeholder under `results/claims/` is not evidence. Four fixed triples must also be rechecked on hardware administered by four distinct external operators.

**Derived tables and the holdout analysis.** The separate registry `audit/baselines.json` records published, archived, and current strict lower bounds with source locators. `tools/close_baselines.py` closes those inputs under the registered recurrences, preserves a provenance path for each output, and generates the tables used in Section 2. The survivor list, reconstructed holdout ledger, and generated audit reproduce the disjoint counts and model means reported in Sections A and B, conditional on the retained event list. These artifacts reproduce the reported derivations; they do not by themselves re-evaluate every prime or establish complete campaign coverage.

**The complete campaign audit.** Claims about exhaustive coverage of  $[9.7 \times 10^8, 2 \times 10^9]$  require the full 515-chunk record. Material that was historically ignored must first be recovered read-only from the campaign PC, including checkpoint evidence for chunk 7, and mapped without inventing unavailable metadata. The release must then establish byte-level identity between the scanner and separate CPU ordered-prime streams on every chunk; agreement of aggregate congruence counts is not a substitute. The four historical scanner blobs and their revision map, together with the preserved aggregate re-scan of the first 17 chunks, are already registered, but neither that re-scan nor the current summaries close the full campaign gate. A complete GPU campaign rerun is required only if recovery reveals a material gap that cannot otherwise be closed.

The cyclic  $W_c$  pilot and its proof artifacts are outside the scientific scope of this records paper and are intentionally absent from this repository. The archived two-colour phase-2 source is credited and located, but a verbatim third-party web capture is not redistributed unless its rights status is resolved. Outbound licences for code, paper, and data also remain an explicit author decision. These rights and provenance records support redistribution and priority attribution; they do not convert an archival coverage statement into a newly audited fact.

After the scientific, campaign, replication, and rights gates pass, the final metadata and DOI will be inserted into the exact release candidate. A checked, immutable tagged archive—not a moving branch—will then be deposited and cited. Until that archive and its external receipt exist, this private repository and manuscript are not represented as a complete citable release.

## A Repository-recorded holdout protocol and evidence boundary

The lower bounds and the holdout comparison have different epistemic status. The former follow from Rabung’s theorem after machine verification of its hypotheses; the latter is empirical and conditional on the count model of Appendix B. This appendix states only what the retained history can reconstruct. It does not elevate that history into an externally timestamped preregistration.

**Retained freeze record.** Chunks 1–180 form the training window for the (3, 17) comparison. At the cut, retained commit `2b5f9f9` records the baseline count and the three numerical intensities of Remark B.1. Its diff already contains one tracker output at chunk 181, so it does not prove that every displayed parameter preceded the first post-cut observation. A later retained commit, ordered before the chunk-250 reading in the preserved Git DAG, records the schedule 250, 300, 400, 515. No later retained revision changes the parameters. The Git objects and remotely visible DAG fix contents and relative order, but their commit dates are not an independent timestamping receipt. We therefore describe this as a *repository-recorded holdout comparison*, not a *preregistration*. The candidate forms and their parameters were selected using the training data and are not called blind.

The retained training record explains  $\eta$  as an empirical training-window level. It fixes  $m = 8 \cdot 10^{-4}$  and  $\beta = 1/2$  but does not recover an exhaustive set of candidate families, a formal objective function, or the number of exploratory trials used to choose those two values. Consequently, the appendix compares three historically fixed curves conditionally; it is not a confirmatory selection test over a prespecified model class.

At each recorded scheduled read the live program compared the cumulative observed increment with the three displayed means, identified the closest displayed mean, and printed Wald-type discrepancies normalized by  $\sqrt{N_{\text{obs}}}$  for two candidates. It was not an exact Poisson test and

did not contain the deviance or dispersion analyses now reported in the paper. Two manually transcribed standardizations for the third candidate were erroneous; the counts and forecast functions were not. The full-interval correction and all later sensitivity calculations are explicitly post hoc and are collected in Appendix C; they do not retroactively become part of the recorded rule. Likewise, retaining fixed  $m$  and  $\beta$  prevents holdout refitting but does not remove their training-window uncertainty; we make no family-wide or long-range extrapolative claim from this comparison.

**One trajectory, four scheduled views.** The four cumulative readings share their early data. Table 7 therefore presents the four disjoint increments as its primary view and retains the cumulative values only as an audit trail. The model-conditional interpretation of those increments is stated with the result in Section B.3.

**Source and predicate checks.** The 515 campaign chunks span eight git revisions and four scanner-source fingerprints. The final fingerprint starts at chunk 79, before the training cut, so the complete post-freeze comparison uses one scanner source. This removes scanner-source changes as a confounder for the recorded holdout comparison; descriptive whole-campaign summaries must nevertheless be accompanied by the full revision table and by tests across the earlier transitions.

The historical campaign journal also reports that 200 sampled run-valid (3, 17) primes were re-tested through a separate CPU discrete-logarithm walk without a reported failure. The present event-list audit does not reproduce that test; its sampled-prime list and complete raw output must be included and hashed in the release before the numerical statement is treated as machine-audited. Even then, the check addresses false survivors, not omissions, prime enumeration, dispersion, or temporal dependence.

**Reproducibility boundary.** The versioned 1859-entry list of reported run-valid (3, 17) primes, labelled complete by the historical analysis, is sufficient to reconstruct all 515 success counts and the disjoint increments in Table 7, conditional on that list. It does not by itself re-evaluate the max-run predicate or prove that no run-valid prime was omitted. The live tracking script additionally reads `results/campaign/checkpoint.json`, while the campaign directory is excluded from ordinary source tracking. An end-to-end archival claim therefore requires an immutable copy of the checkpoint, the raw per-chunk outputs, and a machine-readable revision map in the final data release. Until those files are part of the cited release, the git history alone reproduces the narrow (3, 17) event-count analysis, not every campaign-level diagnostic.

This boundary preserves the reconstructed record as executed, makes later repairs visible as repairs, and limits the conclusions to what the released evidence can support.

## B Holdout check for run-valid (3, 17) primes

The campaign preserved the run-valid survivor event list and aggregate counts, but not the complete per-prime max-run profile. We use those data only to compare three recorded forecasts beyond a fixed training cut. The comparison is not a theorem, a universal density law, or evidence that any Rabung search has been exhausted; every probabilistic statement is conditional on the stated count model.

## B.1 Population and mirror reference

A prime is *run-valid* for  $(r, k)$  when condition (a) holds, that is,  $\text{maxrun}(p, r) < k$ , and *valid* when  $(a) \wedge (b)$  holds. The estimand here is the run-valid population. The bounds of Section 2 use the full criterion. Because condition (b) is not represented in the versioned event list, it cannot be absorbed into the density correction.

The symmetry

$$c(p - x) = c(x) + c(-1) \pmod{r}$$

leaves only the first  $(p - 1)/2$  positions free. The natural i.i.d. reference for the maximal run therefore has length  $n = (p - 1)/2$ , not  $p - 1$ . This descriptively accounts for most of the reference-length shift, but the survivor list does not support a numerical mean-run claim. For  $r = 3$  the case  $c(-1) = r/2$  cannot occur because  $r$  is odd; more generally that case exists only for even  $r$ .

## B.2 Frozen intensity models

The elementary Poisson-clumping reference is

$$\Lambda_{r,k}(p) = \frac{(r - 1)(p - 1)}{2r^k}, \quad g_0(p) = e^{-\Lambda_{r,k}(p)}.$$

For an isolated prime this changes on the scale  $r^k$ . The additional factor  $k$  comes only from the extreme over the available primes. Indeed, with  $a_r = (r - 1)/2$  and fixed  $r$ , the heuristic number of survivors beyond  $x$  is

$$M(x) \approx \int_x^\infty \frac{e^{-a_r t/r^k}}{\varphi(r) \log t} dt \sim \frac{r^k}{a_r \varphi(r) \log x} e^{-a_r x/r^k}.$$

Balancing  $M(x) \asymp 1$  gives  $a_r x/r^k = k \log r - \log k + O_r(1)$ , hence the qualitative extreme-value scale  $x \asymp k r^k$ . We do not use the local fit to predict precise numbers of future primes, declare a well exhausted, or assign narrow uncertainty intervals far outside the fitted window.

The retained program instead used

$$\tilde{\Lambda}_{r,k}(p) = \frac{(r - 1)p}{2r^k}, \quad \tilde{g}_0(p) = e^{-\tilde{\Lambda}_{r,k}(p)},$$

so  $\tilde{\Lambda} - \Lambda = (r - 1)/(2r^k)$ . We retain this convention to reproduce the recorded calculation while keeping  $\Lambda$  as the natural half-length reference.

**Remark B.1** (Domain of the empirical comparison). The two corrected candidates are

$$\tilde{g}_1(p) = e^{-\tilde{\Lambda}_{3,17}(p)} + m e^{-\beta \tilde{\Lambda}_{3,17}(p)}, \quad \tilde{g}_2(p) = e^{-\tilde{\Lambda}_{3,17}(p)} + \eta,$$

with  $m = 8 \cdot 10^{-4}$ ,  $\beta = 1/2$ , and  $\eta = 1.62 \cdot 10^{-5}$ . These are local intensity models for condition (a) in the observed deep-tail window. In particular,  $\tilde{g}_1(p) \rightarrow 1 + m > 1$  as  $\tilde{\Lambda} \rightarrow 0$ , so it is not a global probability law. The parameters were selected on the training window; freezing them prevents holdout tuning but does not remove their estimation uncertainty.

### B.3 Repository-recorded holdout comparison

After chunk 180, when 1727 run-valid  $(3, 17)$  primes had been observed, the retained history records the three candidates  $\tilde{g}_0, \tilde{g}_1, \tilde{g}_2$  under the evidence boundary of Appendix A.

For

$$I_j = [970\,000\,000 + (j - 1) 2\,000\,000, 970\,000\,000 + j 2\,000\,000)$$

and a set  $A$  of chunks, the corrected mean derived from a recorded intensity is

$$\mu_\ell(A) = \sum_{j \in A} \int_{I_j} \frac{\tilde{g}_\ell(p)}{2 \log p} dp, \quad \ell \in \{0, 1, 2\}.$$

The factor  $1/(2 \log p)$  is the analytic density used for primes  $p \equiv 1 \pmod{3}$ . Table 7 reports the repository-recorded scheduled readings as disjoint increments.

Table 7: Repository-recorded scheduled  $(3, 17)$  readings, reported as disjoint increments. The model columns are means integrated over the full half-open chunk intervals.

| chunks  | observed | bare $\tilde{g}_0$ | decaying $\tilde{g}_1$ | constant floor $\tilde{g}_2$ |
|---------|----------|--------------------|------------------------|------------------------------|
| 181–250 | 90       | 68.35              | 80.26                  | 122.20                       |
| 251–300 | 20       | 18.75              | 24.04                  | 57.06                        |
| 301–400 | 16       | 12.58              | 18.59                  | 88.87                        |
| 401–515 | 6        | 2.81               | 5.82                   | 90.04                        |
| total   | 132      | 102.49             | 128.72                 | 358.17                       |

The corresponding cumulative observations are 90, 110, 126, 132. They are four views of one trajectory, not four replications. The fixed decaying forecast is close to the observed total, the recorded constant-floor candidate substantially overpredicts it, and the bare reference underpredicts it. The increments are non-overlapping but are conditionally independent only under the specified inhomogeneous-Poisson model; disjointness alone is not evidence of independence, and a conditional Poisson tail is not a model-robust significance level. Appendix C records the endpoint correction and retrospective sensitivity checks.

### B.4 Summary

The mirror symmetry supplies a useful first-order reference, and one fixed decaying correction tracked the held-out  $(3, 17)$  trajectory more closely than the bare and constant-floor candidates. This is an empirical forecast check for one target and one finite window, not four confirmation datasets, a proof of a universal two-exponential law, or a quantitative certificate that any future Rabung search is empty.

## C Retrospective diagnostics for the holdout comparison

This appendix records corrections and sensitivity analyses performed after the repository-recorded scheduled reads. They test the recorded comparison on its stated finite window; they do not alter the historical rule or enlarge the scope of the empirical claim.

**Remark C.1** (Recorded readout and post hoc diagnostics). Three layers must be kept distinct. First, the recorded inputs were the three intensity functions, their numerical parameters, the baseline count, and—before the first scheduled read—the reading times. Second, the live tool selected the closest displayed mean and printed Wald-type discrepancies  $(N - \mu)/\sqrt{N}$  for the bare and constant-floor candidates. It did not compute deviance residuals or exact Poisson tails; two manually noted standardizations for the decaying candidate were later found to use the wrong denominator.

Third, all deviance, dispersion, autocorrelation, negative-binomial, and block calculations are retrospective. The archival script used chunk midpoints as the two endpoints of a cumulative integral. Table 7 corrects that convention by integrating the full observed half-open chunk intervals. This changes no observation and no recorded operational intensity function, only the integration limits. The corrected means are therefore post-campaign recalculations of the recorded functions. Replacing the operational  $\tilde{\Lambda} = p/3^{17}$  by the natural  $\Lambda = (p - 1)/3^{17}$  changes every holdout total by less than  $9 \cdot 10^{-7}$  and changes no cell of Table 7 at two decimal places.

**Dispersion and dependence.** On the 335 post-cut chunks, the recorded decaying model gives Pearson statistic  $X^2 = 324.2$  (quasi-Poisson scale  $X^2/335 = 0.968$ ); an NB2 sensitivity fit puts the additional-dispersion parameter at the Poisson boundary, and the Poisson deviance is 210.46. The Pearson residuals have Ljung–Box  $Q_{12} = 7.17$  (asymptotic conditional reference  $p = 0.85$ ), and aggregation into blocks of 5, 10, 20 chunks gives Pearson scales 0.81, 0.96, 1.05. These checks reveal no gross overdispersion or serial correlation for that fixed model, but they remain post hoc and have limited power in a sparse, non-stationary tail.

**Block sensitivity of the forecast ordering.** A moving-block resampling of the per-chunk log-score difference (circular blocks of length 10, 20 000 seeded replicates) gives a mean decaying-minus-bare advantage of 0.01185 per chunk, with a 95% percentile interval  $[-0.00509, 0.02957]$  and bootstrap fraction 0.092 at or below zero. For decaying minus the recorded constant-floor model, the corresponding interval is  $[0.31298, 0.49363]$  and no replicate is non-positive. Thus the former comparison is less decisive than the latter. We report the forecast ordering, not a universal fitted family or a discovery-level  $p$ -value.

## Acknowledgements

This work rests on Daniel Monroe’s distributed computation [8] and on the code and direct tally released with it [6], which served as the baseline against which our engine was validated. We are further indebted to the unpublished second phase of that project, whose archived results we recovered, rechecked in historical runs, and report here with credit [7] (Section 2). We thank the maintainers of the Internet Archive, without which those results would have been lost.

## Declaration on artificial-intelligence assistance

The research was organized by the author with substantial assistance from artificial-intelligence systems—Anthropic’s Claude (models `claude-fable-5` and `claude-opus-4-8`) and OpenAI Codex in ChatGPT Work—used to explore and design the computational method, plan and analyse the campaign, draft and revise the manuscript, examine the literature and provenance, and carry out separate counter-audits. Under the author’s direction, these systems created

or substantially developed all original computational code for the project: the GPU scanner, campaign drivers, reference implementations and CPU verifiers, statistical- and audit-analysis programs, recovery and operator tooling, tests, and reproducibility and release tooling.

The sole exception among computation code used or consulted during the project is pre-existing code recovered from Daniel Monroe’s project. It is treated as an attributed upstream source, its provenance is tracked, and it is not redistributed in this repository; no claim of authorship is made for it.

These procedures constitute neither an independent scientific validation nor a human certification of the computations. The author conceived the project, set its scope and claims, provided and operated the hardware, and alone assumes responsibility for the manuscript, the code and data attributed to this project, the references, the conclusions, and their possible errors and limitations.

## References

- [1] Thomas Blankenship, Jay Cummings, and Vladislav Taranchuk. A new lower bound for van der Waerden numbers. *European Journal of Combinatorics*, 69:163–168, 2018. doi: 10.1016/j.ejc.2017.10.007. URL <https://doi.org/10.1016/j.ejc.2017.10.007>.
- [2] Marcelo Campos, Jacob Fox, and Carl Schildkraut. A new lower bound for two-color van der Waerden numbers. arXiv:2608.20824v1, August 2026. URL <https://arxiv.org/abs/2608.20824v1>. Version 1, 21 August 2026.
- [3] Jacob Fox and Zach Hunter. Three-color van der Waerden numbers grow super-exponentially. arXiv:2606.02541v1, June 2026. URL <https://arxiv.org/abs/2606.02541v1>. Version 1, 1 June 2026.
- [4] P. R. Herwig, M. J. H. Heule, P. M. van Lambalgen, and H. van Maaren. A new method to construct lower bounds for van der Waerden numbers. *The Electronic Journal of Combinatorics*, 14(1):R6, 2007. doi: 10.37236/925. URL <https://doi.org/10.37236/925>.
- [5] Meilian Liang, Xiaodong Xu, Zehui Shao, and Baoxin Xiu. Lower bounds on some van der Waerden numbers based on quadratic residues. *Journal of Combinatorial Mathematics and Combinatorial Computing*, 83:23–31, 2012. URL <https://combinatorialpress.com/jcmcc-articles/volume-083/lower-bounds-on-some-van-der-waerden-numbers-based-on-quadratic-residues/>.
- [6] Daniel Monroe. Direct Rabung tally distributed with the van der Waerden search code. Versioned project data file, 2021. URL <https://github.com/hmonroe/vdw/blob/f1223097f4adc0a41ccb92ace51ebafd2b996fb1/output.txt>. Immutable Git commit f1223097f4adc0a41ccb92ace51ebafd2b996fb1; file output.txt; SHA-256 bd88e781acdceead847e0b736ca788cbc510aac744d025c2efbde4b152725774.
- [7] Daniel Monroe. BOINC phase-2 bounds table. Internet Archive capture of <http://vdwnumbers.org/bounds.txt>, 2021. URL [https://web.archive.org/web/20220428101631id\\_/http://vdwnumbers.org/bounds.txt](https://web.archive.org/web/20220428101631id_/http://vdwnumbers.org/bounds.txt). Source header dated 18 August 2021; archived 28 April 2022; SHA-256 0e8f79232e3855cbc64026a4c2f3a82433c0508456e0c23f76eb1a400efeb556.

- [8] Daniel Monroe. New lower bounds for Van der Waerden numbers using distributed computing. *Journal of Combinatorial Mathematics and Combinatorial Computing*, 128:305–315, 2026. doi: 10.61091/jcmcc128-19. URL <https://doi.org/10.61091/jcmcc128-19>. Published online 4 December 2025.
- [9] Daniel Monroe. Projects. Personal website, <https://daniel-monroe.github.io/projects.html>, 2026. URL <https://github.com/daniel-monroe/daniel-monroe.github.io/blob/825778cca5e669cbf60140e35170113603010076/projects.html>. Archived author-owned version dated 28 March 2026; live page accessed 28 August 2026.
- [10] Tomás Oliveira e Silva. Tables of values of  $\pi(x)$  and of  $\pi_2(x)$ . University of Aveiro; data file <https://sweet.ua.pt/tos/primes/1d09.txt.gz>, 2005. URL [https://web.archive.org/web/20160704031247id\\_/http://sweet.ua.pt/tos/primes/1d09.txt.gz](https://web.archive.org/web/20160704031247id_/http://sweet.ua.pt/tos/primes/1d09.txt.gz). Table of  $\pi(k \cdot 10^9)$  last updated 23 January 2005; accessed 28 August 2026.
- [11] John Rabung and Mark Lotts. Improving the use of cyclic zippers in finding lower bounds for van der Waerden numbers. *The Electronic Journal of Combinatorics*, 19(2):P35, 2012. doi: 10.37236/2363. URL <https://doi.org/10.37236/2363>.
- [12] John R. Rabung. Some progression-free partitions constructed using Folkman’s method. *Canadian Mathematical Bulletin*, 22(1):87–91, 1979. doi: 10.4153/CMB-1979-012-1. URL <https://doi.org/10.4153/CMB-1979-012-1>.
- [13] Jonathan Sorenson and Jonathan Webster. Strong pseudoprimes to twelve prime bases. *Mathematics of Computation*, 86(304):985–1003, 2017. doi: 10.1090/mcom/3134. URL <https://doi.org/10.1090/mcom/3134>.
- [14] Xiaodong Xu. A generalization of sets without long arithmetic progressions based on Szekeres algorithm. *Journal of Number Theory*, 133(11):3670–3677, 2013. doi: 10.1016/j.jnt.2013.05.008. URL <https://doi.org/10.1016/j.jnt.2013.05.008>.